



ΠΑΝΕΠΙΣΤΗΜΙΟ
ΠΑΤΡΩΝ
UNIVERSITY OF PATRAS

2023- 2024

Εργαστηριακή άσκηση Μέρος Β

ΜΠΑΡΑΚΟΥ ΓΕΩΡΓΙΑ

ΑΜ : 1090075

4ο έτος σπουδών

Περιεχόμενα

Θέμα 1ο : Ταξινόμηση εικόνων.....	2
Μέρος Α : Κατηγοριοποίηση Εικόνων με χρήση Συνελικτικών Νευρωνικών Δικτύων (CNN)	2
Περιγραφή του μοντέλου	2
Ερώτημα 1	2
Ερώτημα 2	3
Ερώτημα 3	3
Ερώτημα 4	4
Ερώτημα 5	5
Μέρος Β : Κατηγοριοποίηση Εικόνων μέσω εξαγωγής χαρακτηριστικών και νευρωνικού ταξινομητή.	7
Ερώτημα 1	7
Ερώτημα 2	7
Ερώτημα 3	10
Κώδικας:	11
Μέρος Α : main.py.....	11
Μέρος Β : extract_features.py	15
Μέρος Α αποθήκευση καλύτερου δικτύου : BestModel.py.....	18

Θέμα 1ο : Ταξινόμηση εικόνων

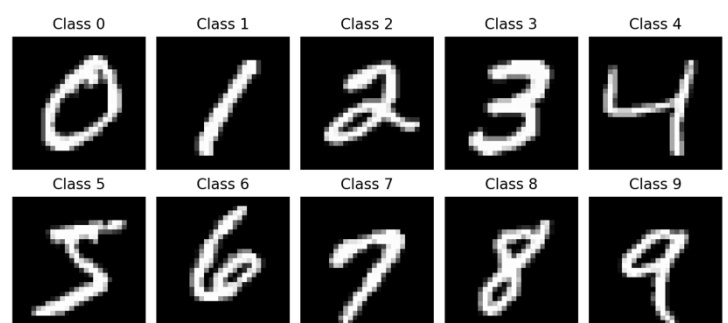
Μέρος Α : Κατηγοριοποίηση Εικόνων με χρήση Συνελικτικών Νευρωνικών Δικτύων (CNN)

Περιγραφή του μοντέλου

Αρχικά, τα δεδομένα εισόδου, δηλαδή οι εικόνες φορτώνονται και κάθε εικόνα μετατρέπεται σε tensor (πολυδιάστατος πίνακας αριθμών) και κανονικοποιείται (τιμές των pixel σε συγκεκριμένο εύρος). Στο convolution-1 που περνούν οι εικόνες εισόδου, έχουμε 6 φίλτρα 5×5 που εφαρμόζονται στην κάθε εικόνα και κάθε φίλτρο παράγει στην έξοδο 6 διαφορετικά κανάλια χαρακτηριστικών. Έπειτα, αυτά τα αποτελέσματα περνούν από μία συνάρτηση ενεργοποίησης ReLU, για την εισαγωγή μη-γραμμικότητας. Στην συνέχεια, έχουμε το pooling, η διαδικασία της μείωσης των διαστάσεων 2×2 (max-pooling). Στην επόμενη φάση, convolution-2, εφαρμόζονται 12 φίλτρα 5×5 στα κανάλια χαρακτηριστικών και παράγονται καινούργια. Η ReLU ξαναχρησιμοποιείται και τέλος 2^ο pooling 2×2 (max-pooling). Μέχρι και αυτό το σημείο είναι η διαδικασία της εξαγωγής των χαρακτηριστικών. Συνεχίζει, στην ταξινόμηση των χαρακτηριστικών, όπου κάνει vectorization, δηλαδή τα δεδομένα (τα χαρακτηριστικά) που παράχθηκαν μετατρέπονται σε διανύσματα για να εισαχθούν στα fully connected layers, όπου η έξοδος τους (output 10) θα αντιπροσωπεύει την κάθε κλάση, 0 έως 9 αριθμούς που περιέχονται στο MNIST dataset.

Ερώτημα 1

Αρχικά, ζητήθηκε η απεικόνιση δειγμάτων από κάθε κλάση. Οι κλάσεις είναι συνολικά 10 και η κάθε μία αντιπροσωπεύει μία εικόνα με έναν αριθμό από 0 έως 9. Η συνάρτηση διατρέχει από κάθε κλάση και την εισάγει σε ένα figure.



Ερώτημα 2

Στην συνέχεια, χωρίστηκε το σύνολο των δεδομένων (60.000 δείγματα) για την εκπαίδευση και για την επικύρωση, με ποσοστά 80% και 20% αντίστοιχα. Τα δεδομένα εκπαίδευσης χρησιμοποιούνται για την εκπαίδευση του μοντέλου, την ενημέρωση των βαρών και το loss, ενώ τα δεδομένα επικύρωσης χρησιμοποιούνται για τον έλεγχο της απόδοσης του μοντέλου, δηλαδή ελέγχουν πόσο “καλή” εκπαίδευση πραγματοποιήθηκε στο μοντέλο.

```
# train : 80% , validation : 100 - 80 = 20%
training = int(0.8 * len(dataset))
validation = len(dataset) - training
training_subset, validation_subset = random_split(dataset, [training, validation])

print(f"Training: {len(training_subset)}")
print(f"Validation: {len(validation_subset)}")
```

Training: 48000
Validation: 12000

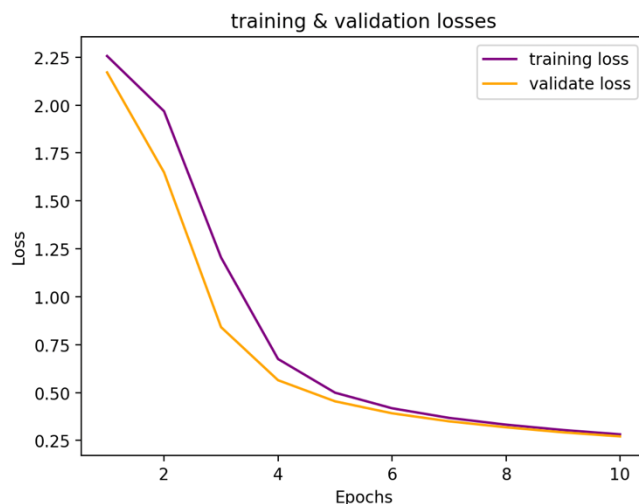
Ερώτημα 3

Με την χρήση του αλγορίθμου Stochastic Gradient Descent (SGD), για κάθε epoch, τα δεδομένα εκπαίδευσης χωρίζονται σε mini-batches, όπου το κάθε mini-batch περιέχει κάποια δείγματα. Αφού “περνούν” μέσα από το μοντέλο τα mini-batches, θα έχουμε τις προβλέψεις. Από τις προβλέψεις και τις πραγματικές “ετικέτες”, όπου είναι οι αριθμοί από 0-9, θα υπολογιστεί η απώλεια. Έπειτα, θα υπολογιστούν τα gradients των απωλειών και στο τέλος ο optimizer (SGD) θα ενημερώσει (learning_rate) τα βάρη του μοντέλου με βάση τα gradients. Έπειτα, για την διαδικασία της επικύρωσης, τα βάρη των μοντέλων δεν θα ενημερώνονται και για κάθε mini-batch θα υπολογίζεται η απώλεια.

Στο διπλανή γραφική παρατηρούνται τα σφάλματα εκπαίδευσης και τα σφάλματα επικύρωσης. Γενικότερα, είναι γνωστό ότι στην διαδικασία εκπαίδευσης μπορεί το μοντέλο να “μάθει” για κάτι, αλλά με μεγάλη λεπτομέρεια (υπερ-εκπαίδευση) και να μην μπορεί να το γενικεύσει. Έτσι, το μοντέλο “μαθαίνει” περισσότερα απ’ ότι θα έπρεπε με αποτέλεσμα όταν θα πραγματοποιείται η διαδικασία επικύρωσης να μην μπορεί να “καταλάβει” από τα δεδομένα. Συνήθως αυτό γίνεται όταν η καμπύλη του σφάλματος

επικύρωσης από εκεί που έχει “σταθεροποιηθεί”, θα αρχίσει να ανεβαίνει.

Παρατηρούμε ότι όσο περνάνε τα epoch τόσο μικρότερο σφάλμα έχουμε και ότι και οι δύο καμπύλες αρχίζουν να σταθεροποιούνται. Καλό είναι να σταματάει η εκπαίδευση σε αυτό περίπου το σημείο ώστε να μην αυξηθεί μετά το σφάλμα επικύρωσης.



```

Training loss decreased from 0.3680 to 0.3326.
Validation loss decreased from 0.3502 to 0.3190.
Training epoch: 9 [0/48000 (0%)] loss: 0.267887
Training epoch: 9 [6400/48000 (13%)] loss: 0.432208
Training epoch: 9 [12800/48000 (27%)] loss: 0.326615
Training epoch: 9 [19200/48000 (40%)] loss: 0.316015
Training epoch: 9 [25600/48000 (53%)] loss: 0.259007
Training epoch: 9 [32000/48000 (67%)] loss: 0.257400
Training epoch: 9 [38400/48000 (80%)] loss: 0.203453
Training epoch: 9 [44800/48000 (93%)] loss: 0.219746

AV loss: 0.2926

Training loss decreased from 0.3326 to 0.3048.
Validation loss decreased from 0.3190 to 0.2926.
Training epoch: 10 [0/48000 (0%)] loss: 0.340592
Training epoch: 10 [6400/48000 (13%)] loss: 0.247774
Training epoch: 10 [12800/48000 (27%)] loss: 0.327658

```

Έπειτα, ορίστηκε ένα αρχικό “loss” με τιμή πολύ μεγάλη (inf), το οποίο συγκρίνεται με του πρώτου epoch την απώλεια. Έτσι κάθε φορά που “περνάει” ένα νέο epoch θα συγκρίνεται η απώλεια του με του προηγούμενου. Στην ουσία δηλαδή θα συγκρίνεται η απόδοση. Παρατηρείται ότι από το epoch 9 μέχρι το epoch 10 μειώθηκε το σφάλμα, και άρα αυξήθηκε η απόδοση.

Το τελικό δίκτυο είναι αυτό που θα έχει

το μικρότερο loss σε εκπαίδευση και επικύρωση, το οποίο και αποθηκεύουμε :

best_training.pth και best_validate.pth και ανακτάμε/ εκτυπώνουμε στο BestModel.py

```

Conv1.weight: tensor([[[[-2.1506e-02, -4.2110e-02, -1.4545e-01, -2.1290e-01, -1.4
[ 1.4416e-01, 2.6894e-02, -2.3737e-01, -4.7604e-02, 1.8345e-01],
[ 1.9145e-02, -2.3274e-01, -8.4490e-02, 1.8634e-01, 1.7174e-01],
[ 1.3009e-02, -8.9834e-02, 1.7359e-01, -1.2968e-01, 2.8296e-01],
[-9.0455e-02, -1.0757e-01, 1.8279e-02, -4.6439e-02, 1.8916e-01]]],
[[[-2.6323e-01, -3.6102e-01, -3.2319e-01, -2.0113e-01, -3.6849e-01],
[-2.2775e-01, -2.8678e-01, -2.1234e-01, -2.2431e-01, -3.6544e-01],
[-8.6785e-02, -4.0917e-01, -3.5654e-01, -2.9519e-01, -1.9334e-01],
[-2.8213e-01, -6.5234e-02, -3.1734e-01, -2.7030e-01, -6.7647e-02],
[-2.2018e-01, -2.1099e-01, -3.0475e-01, -2.1948e-01, -1.0115e-01]]],
[[[-1.7465e-01, -1.0382e-01, 1.4008e-01, 1.3976e-01, 1.2245e-01],
[ 9.1845e-02, 1.5981e-01, 1.6131e-02, 2.3206e-01, 1.3839e-01],
[ 1.9430e-01, -8.3791e-02, -1.6601e-01, -2.9301e-02, 1.3598e-01],
[ 6.4010e-03, 1.6736e-01, 5.4923e-02, -7.7983e-02, 2.0260e-01],
[ 1.5916e-01, 1.9643e-01, -2.7760e-02, -1.0746e-01, 9.0012e-02]]],

```

Ερώτημα 4

Ο υπολογισμός της ακρίβειας του δικτύου πραγματοποιείται ως εξής : Αρχικά το μοντέλο τίθεται σε λειτουργία evaluation, ώστε να μην λειτουργούν οι λειτουργίες του training όπως π.χ. η ανανέωση βαρών, όπως επίσης και ο υπολογισμός των gradients. Έπειτα, αρχικοποιήθηκαν δύο λίστες όπου θα εισάγονται οι προβλέψεις και οι πραγματικές τιμές του μοντέλο, και ορίστηκε μεταβλητή που θα μετράει πόσες από τις προβλέψεις ήταν σωστές. Έπειτα, για κάθε εικόνα και αντίστοιχη τιμή (ετικέτα) σε κάθε batch μεταφέρονται στο device. Αφού περάσουν τα δεδομένα στο μοντέλο και παραχθούν οι προβλέψεις τους, γίνεται σύγκριση με βάση τα πραγματικά. Για κάθε σωστή πρόβλεψη αυξάνεται η μεταβλητή που μετράει. Στο τέλος, ο υπολογισμός της ακρίβειας είναι οι σωστές προβλέψεις / συνολικός αριθμός των data που πέρασαν από το μοντέλο.

```

Accuracy of the Neural Network: 93.16%
Accuracy: 0.9316

```

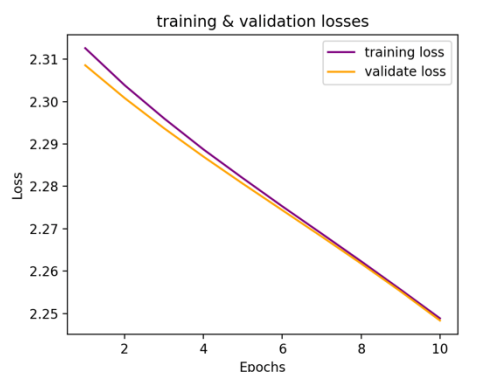
Γίνεται εκτύπωση και του confusion matrix (γραμμή : πραγματικές τιμές, στήλες : προβλέψεις), το οποίο δείχνει σε κάθε κελί του τον αριθμό των δειγμάτων που προβλέφθηκαν σωστά.

```
Confusion Matrix:
[[ 961    1    6    1    0    2    6    1    2    0]
 [    0 1110    4    3    2    1    7    0    8    0]
 [   15    4  916   16   21    2   11   21   22    4]
 [    3    4   17  948    0   10    1    7   14    6]
 [    0    2    3    0  949    0    9    1    3   15]
 [    5    3    2   21    3  821   16    3   11    7]
 [   14    8    3    0   11   16  903    0    3    0]
 [    4   11   31    7    3    1    0  935    3   33]
 [    5   11   10   21    9   12    7   15  867   17]
 [    9    7   12    6   25   15    0   18   11  906]]
```

Ερώτημα 5

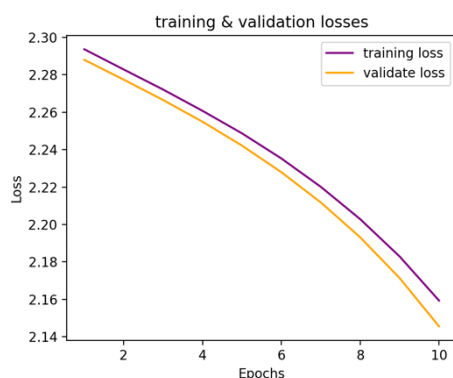
Πραγματοποιείται η επανάληψη όλης της παραπάνω διαδικασίας, αλλά αντί για ποσοστά εκπαίδευσης – επικύρωσης 80%-20% αντίστοιχα, θα έχουμε για το 5%, 10%, 50% και 100% της εκπαίδευσης.

Αρχικά για το 5% της εκπαίδευσης, παρατηρούμε:



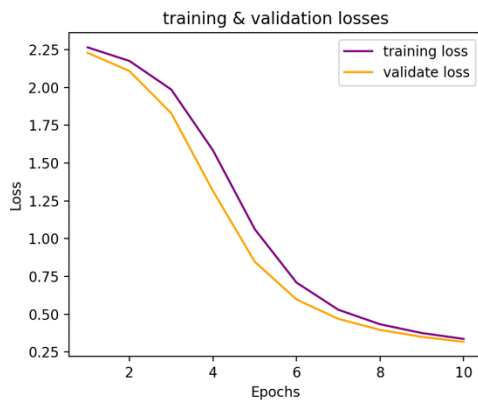
```
Accuracy of the Neural Network: 23.28%
Accuracy: 0.2328
Confusion Matrix:
[[335    0   14   45    2   27 388    0  167    2]
 [    0    0   175    0    1  439    0  520    0]
 [   61    2   83   47   19   13 376    3  417   11]
 [   14    0    8  188   16   65 412    0  298    9]
 [   28    0  222   56  106   39   81    0  441    9]
 [   11    1    4   92    6  135 389    0  252    2]
 [   33    0   37    6   15   17 786    0   58    6]
 [    5    3   11  181  194  273    5   20  296   40]
 [   10    0    3   39   13   10 232    0  667    0]
 [   30    0   25  133   83   25   22    0  683    8]]
```

Για το 10% :



```
Accuracy of the Neural Network: 42.97%
Accuracy: 0.4297
Confusion Matrix:
[[ 523    84    8   13   85  101    0  151    0   15]
 [    1  1129    0    0    3    1    0    1    0    0]
 [   17  315  326   14  191    9    2  129   11   18]
 [   34  659    0  103   39   97    6   25   27   20]
 [    0  109    9    3  690    6    0   12    0  153]
 [   49  406    1    6  175  173    1   32    1   48]
 [    4  192   20    4  666   33    1    9    1   28]
 [    2  163    7    0   16    4    0  802    0   34]
 [    2  399    1    5  312   29    0   44   62  120]
 [    1  205    1    2  169   15    0  128    0  488]]
```

Για το 50% :

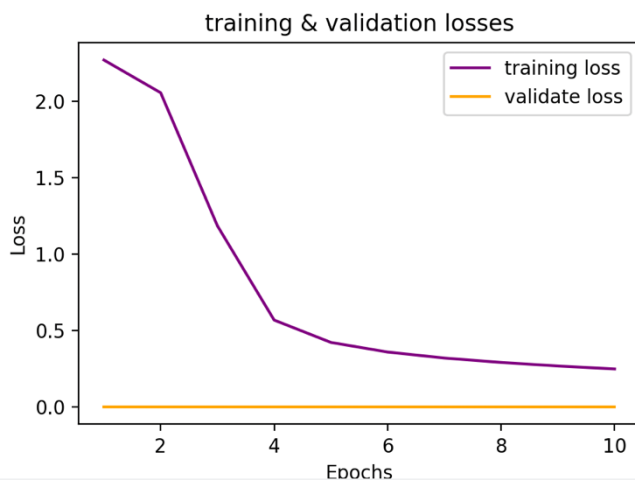


```

Accuracy of the Neural Network: 91.82%
Accuracy: 0.9182
Confusion Matrix:
[[ 954  0  3  1  0  4  12  1  5  0]
 [  0 1104  5  4  3  1  3  0  15  0]
 [  9  3  925  7  15  6  21  22  24  0]
 [  1  5  22  926  0  16  2  12  18  8]
 [  0  2  2  0  924  0  22  1  3  28]
 [ 10  3  4  46  3  784  19  5  10  8]
 [ 18  7  7  0  5  12  907  0  2  0]
 [  5  15  37  2  4  1  0  902  13  49]
 [  8  7  3  31  11  16  9  18  853  18]
 [  8  9  10  10  20  11  0  22  16  903]]

```

Τέλος, για το 100% :



```

Accuracy of the Neural Network: 93.46%
Accuracy: 0.9346
Confusion Matrix:
[[ 966  0  2  0  0  0  5  1  6  0]
 [  0 1110  4  3  1  2  2  0  13  0]
 [ 12  3  948  10  10  0  11  19  14  5]
 [  6  2  18  928  0  13  1  14  20  8]
 [  0  3  3  0  914  0  15  1  4  42]
 [ 12  4  4  23  2  806  11  6  20  4]
 [ 17  5  2  0  6  11  913  0  4  0]
 [  2  15  27  6  4  2  0  943  6  23]
 [ 11  6  4  15  8  12  4  11  889  14]
 [ 10  7  9  6  12  5  0  16  15  929]]

```

Γενικότερα, είναι γνωστό ότι το μοντέλο χρειάζεται αρκετά δεδομένα για εκπαίδευση, έτσι ώστε να αντιλαμβάνεται λεπτομέρειες κτλ. Όμως, τα υπερβολικά δεδομένα εκπαίδευσης οδηγούν σε υπερ-εκπαίδευση, που όπως αναλύθηκε στο ερώτημα 3, δεν είναι καλό για το μοντέλο. Για το 5% βλέπουμε πως η ακρίβεια είναι στο 23%, κάτι που σημαίνει ότι δεν έχουν δοθεί αρκετά δεδομένα για την εκπαίδευση του μοντέλου. Για το 100% παρατηρείται 93,46% ακρίβεια, το οποίο είναι καλό. Όμως, το dataset του MNIST είναι σχετικά μικρό, οπότε δεν γνωρίζουμε τι θα γινόταν εάν δίνονταν στο μοντέλο άγνωστα δεδομένα. Συμπερασματικά, το μοντέλο χρειάζεται αρκετά δεδομένα για εκπαίδευση, αλλά σίγουρα θα χρειαστεί και δεδομένα για την επικύρωση, επειδή, έτσι γνωρίζουμε πόσο καλά πήγε η εκπαίδευση του. Για το 80-20 είχαμε ακρίβεια στο περίπου 93,16%, το οποίο είναι αρκετά καλό ποσοστό.

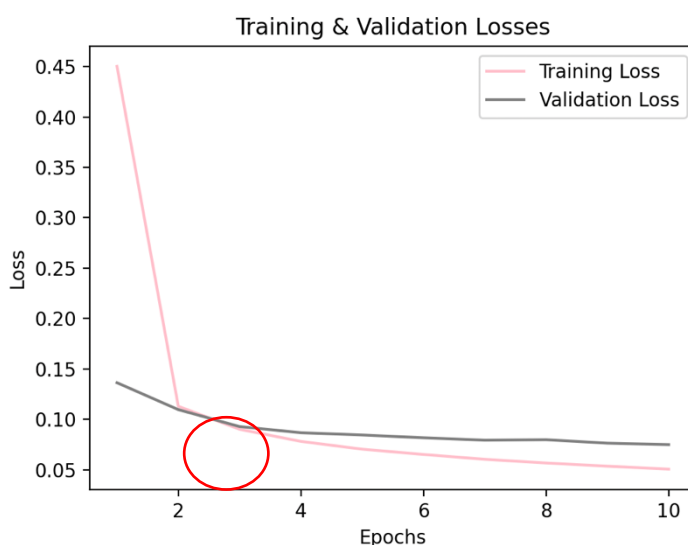
Μέρος B : Κατηγοριοποίηση Εικόνων μέσω εξαγωγής χαρακτηριστικών και νευρωνικού ταξινομητή.

Ερώτημα 1

Για την εξαγωγή των χαρακτηριστικών, μετατρέπουμε την κάθε εικόνα σε numpy array και έτσι μέσω του Histogram of Oriented Gradients (HOG) υπολογίζονται τα χαρακτηριστικά της κάθε εικόνας. Ο αλγόριθμος HOG χωρίζει την κάθε εικόνα σε cells των 4×4 pixels και μετά τα cells συνδυάζονται σε block των 2×2 cells και στο τέλος επιστρέφεται η εικόνα HOG με τα χαρακτηριστικά της αντίστοιχης εικόνας. Στο συγκεκριμένο μοντέλο (αφού αφαιρέθηκε η διαδικασία της συνέλιξης), θα εισάγονται τα χαρακτηριστικά/ δεδομένα HOG και να έχει ως έξοδο της 10 κατηγορίες (0-9). Με τον ίδιο τρόπο έχουμε πάλι τα δεδομένα training / εκπαίδευσης και τα δεδομένα validate / επικύρωσης.

Ερώτημα 2

Αρχικά, με τον ίδιο τρόπο χωρίσαμε τα δεδομένα εκπαίδευσης και επικύρωσης σε 80-20 αντίστοιχα. Η διαδικασία της εκπαίδευσης, λειτουργεί τον ίδιο τρόπο όπως και πριν, αλλά αυτή την φορά εισάγονται ως είσοδοι τα (διανύσματα) χαρακτηριστικά HOG. Υπολογίστηκαν τα σφάλματα για κάθε epoch και άρα και η απόδοση τους και αποθηκεύτηκε το δικτύου με την καλύτερη απόδοση. Παρακάτω φαίνεται η γραφική με τα σφάλματα εκπαίδευσης, επικύρωσης ανά epoch. Παρατηρείται ότι η καμπύλη της επικύρωσης ανέρχεται πάνω από της εκπαίδευσης περίπου στο epoch 3, αυτό σημαίνει ότι γίνεται υπερ-εκπαίδευση από εκείνο το σημείο και μετά. Άρα γύρω στο epoch 3 το μοντέλο έχει εκπαιδευτεί πιο σωστά.



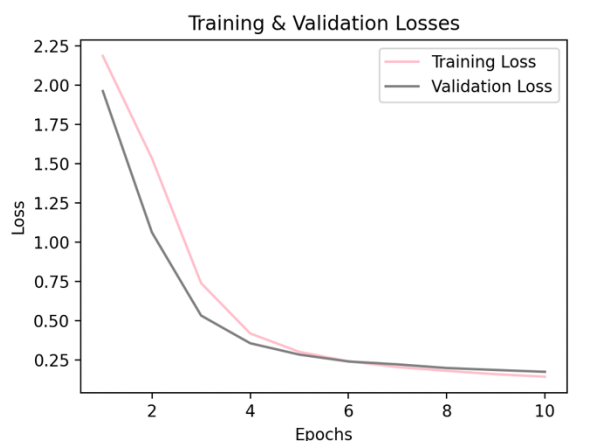
```
Train Features Shape: (60000, 1296)
Test Features Shape: (10000, 1296)
Epoch [1/10], Train Loss: 0.4512
Validation Loss: 0.1404
Epoch [1/10], Validation Loss: 0.1404
Epoch [2/10], Train Loss: 0.1125
Validation Loss: 0.1069
Epoch [2/10], Validation Loss: 0.1069
Epoch [3/10], Train Loss: 0.0894
Validation Loss: 0.0978
Epoch [3/10], Validation Loss: 0.0978
Epoch [4/10], Train Loss: 0.0777
Validation Loss: 0.0898
Epoch [4/10], Validation Loss: 0.0898
Epoch [5/10], Train Loss: 0.0700
Validation Loss: 0.0866
Epoch [5/10], Validation Loss: 0.0866
Epoch [6/10], Train Loss: 0.0643
Validation Loss: 0.0822
Epoch [6/10], Validation Loss: 0.0822
Epoch [7/10], Train Loss: 0.0598
Validation Loss: 0.0805
Epoch [7/10], Validation Loss: 0.0805
Epoch [8/10], Train Loss: 0.0561
Validation Loss: 0.0790
Epoch [8/10], Validation Loss: 0.0790
Epoch [9/10], Train Loss: 0.0530
Validation Loss: 0.0779
Epoch [9/10], Validation Loss: 0.0779
Epoch [10/10], Train Loss: 0.0501
Validation Loss: 0.0756
Epoch [10/10], Validation Loss: 0.0756
Best Validation Loss: 0.0756
```


Στην συνέχεια υπολογίστηκε η ακρίβεια του δικτύου και το confusion matrix.
Παρατηρείται 98.05% ακρίβεια.

```
Accuracy of the Neural Network: 98.05%
Confusion Matrix:
[[ 969  0  3  0  1  3  1  2  1]
 [  1 1124  1  1  0  1  4  1  2  0]
 [  3  2 1016  2  0  0  2  2  5  0]
 [  0  0  2 996  0  5  0  2  3  2]
 [  1  2  1  0 958  0  2  2  4 12]
 [  2  0  2  8  0 876  1  0  3  0]
 [  3  2  0  0  1  5 944  0  3  0]
 [  0  4  8  4  5  0  0 997  3  7]
 [  3  0  4  6  2  1  2  4 946  6]
 [  1  4  1  3  6  3  0  6  6 979]]
```

Πραγματοποιείται η επανάληψη όλης της παραπάνω διαδικασίας, αλλά αντί για ποσοστά εκπαίδευσης – επικύρωσης 80%-20% αντίστοιχα, θα έχουμε για το 5%, 10%, 50% και 100% της εκπαίδευσης.

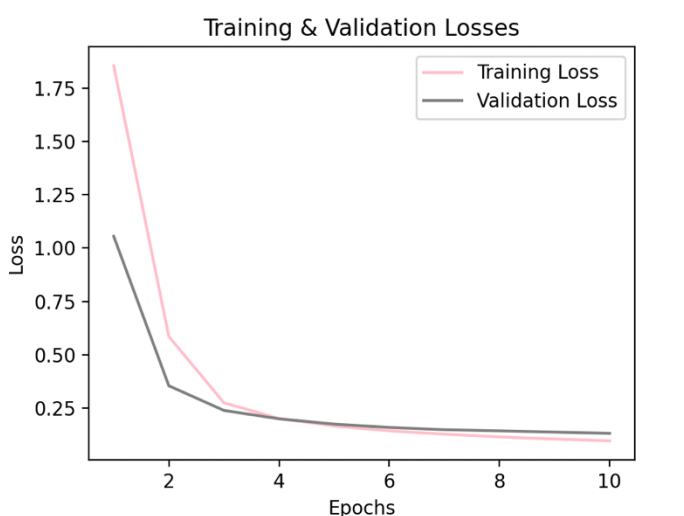
Αρχικά για το 5% της εκπαίδευσης, παρατηρούμε:



```
Accuracy of the Neural Network: 95.84%
Confusion Matrix:
[[ 965  0  3  0  0  2  7  1  2  0]
 [  0 1121  7  0  0  0  3  0  4  0]
 [  9  4 973 11  3  0  3  9 20  0]
 [  0  0  3 974  0 15  1  5  9  3]
 [  1  6  2  0 953  0  4  0  3 13]
 [  2  0  2 15  0 861  6  0  5  1]
 [  9  2  0  0  4 12 929  0  2  0]
 [  1  9 19  0  7  0  0 967  3 22]
 [ 17  4 10 11  5  7  2  6 901 11]
 [  9  5  2  9 18  2  0 15  9 940]]
```

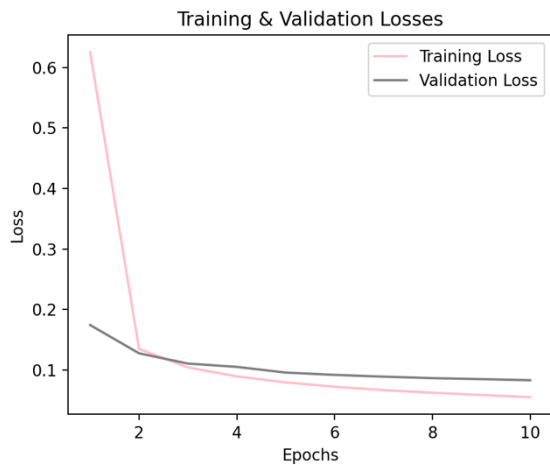
Στην epoch 6 σταθεροποιούνται οι καμπύλες μαζί και άρα έχουμε την καλύτερη εκπαίδευση. Από εκεί και μετά παρατηρούμε την αρχή της υπερ-εκπαίδευσης.

Για το 10%, φαίνεται υπέρ-εκπαίδευση από το epoch 5 και μετά :



```
Accuracy of the Neural Network: 96.65%
Confusion Matrix:
[[ 962  0  6  0  0  2  6  1  3  0]
 [  1 1121  5  1  1  1  3  0  2  0]
 [  5  6 997  1  2  0  3  7 11  0]
 [  0  0  2 980  0  9  0  8  8  3]
 [  1  4  3  0 955  0  1  2  2 14]
 [  2  1  1 15  0 857  5  0  9  2]
 [ 10  2  0  0  4  8 929  1  4  0]
 [  1  6 13  0  3  0  0 991  4 10]
 [ 12  2  8  7  4  4  2  6 915 14]
 [  4  4  3  4 14  2  0 11  9 958]]
```

Για το 50% :

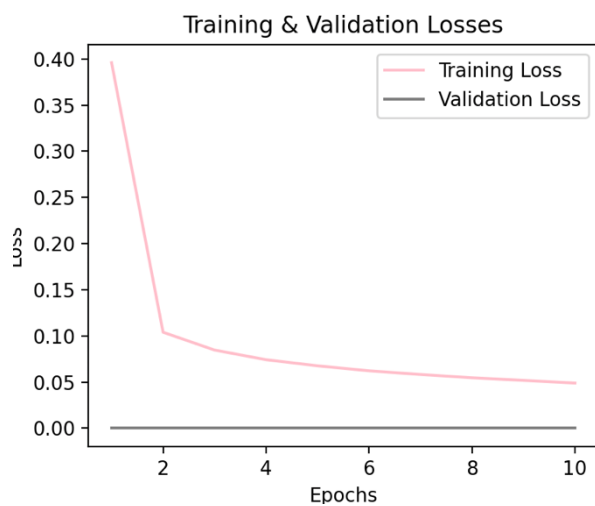


Accuracy of the Neural Network: 97.64%

Confusion Matrix:

[968	0	2	0	0	2	3	1	3	1]
[1 1125	1	1	0	1	3	1	2	0]	
[3 7 1006	1	0	0	2	3	10	0]		
[0 0 2 988	0	9	0	2	7	2]			
[1 2 2 0 956	0	3	2	4	12]				
[2 0 2 7 0 876	3	0	2	0]					
[5 2 0 0 2 6 942	0	1	0]						
[0 5 12 1 4 0 0 995	3	8]							
[6 1 5 6 2 2 3 4 939	6]								
[2 4 2 4 8 4 0 10 6 969]	]								

Τέλος, για το 100% :



Accuracy of the Neural Network: 98.11%

Confusion Matrix:

[973	0	1	0	0	1	2	1	1	1]
[1 1124	1	1	1	1	3	1	2	0]	
[3 5 1013	2	0	0	2	2	5	0]		
[0 1 3 987	0	5	0	4	9	1]			
[1 0 0 0 968	0	2	1	2	8]				
[2 1 2 7 0 874	2	0	4	0]					
[3 1 0 0 1 3 949	0	1	0]						
[0 6 6 2 4 0 0 1000	3	7]							
[4 0 2 4 2 1 3 4 949	5]								
[1 3 1 2 13 3 0 7 5 974]	]								

Ερώτημα 3

Για ίδια δεδομένα εκπαίδευσης – επικύρωσης, 80-20 αντίστοιχα και στα δύο μέρη, παρατηρούμε ότι στο μέρος A η γραφική των losses ανά epoch, σταθεροποιείται προς τα τελευταία epoch ενώ στο μέρος B αρχίζει να κάνει υπέρ-εκπαίδευση πολύ πιο γρήγορα. Στο μέρος B, με την εξαγωγή και χρήση χαρακτηριστικών στο μοντέλο γίνεται πολύ πιο γρήγορα η εκπαίδευση ενώ από την άλλη στο μέρος A χρειάζεται περισσότερο χρόνο. Ακόμη, παρατηρούμε ότι το μέρος A έχει καλύτερη απόδοση (μικρότερο loss) απ' ότι το μέρος B. Άρα, καταλήγουμε στο ότι εάν τα δεδομένα είναι πιο λίγα και λιγότερο πολύπλοκα αρκεί η διαδικασία του μέρους B για το μοντέλο, ενώ εάν έχουμε πολύ περισσότερα δεδομένα είναι προτιμότερο να κάνουμε χρήση της διαδικασίας του μέρους A.

Κώδικας:

Μέρος A : main.py

```
import torch
import torch.nn as nn
import torch.optim as optim
import torchvision.transforms as transforms
import torchvision.datasets as datasets
from torch.utils.data import DataLoader, random_split
import torch.nn.functional as F
import matplotlib.pyplot as plt
from sklearn.metrics import confusion_matrix
import ssl

ssl._create_default_https_context = ssl._create_unverified_context

batch_size = 64
learning_rate = 0.001
num_epochs = 10

class NeuralNetwork(nn.Module):
    def __init__(self):
        super(NeuralNetwork, self).__init__()
        self.Conv1 = nn.Conv2d(in_channels=1, out_channels=6,
kernel_size=5, padding=0)
        self.pooling = nn.MaxPool2d(kernel_size=2, stride=2)
        self.Conv2 = nn.Conv2d(in_channels=6, out_channels=12,
kernel_size=5, padding=0)
        self.fullyConnected1 = nn.Linear(in_features=12*4*4,
out_features=10)
    def forward(self, x):
        x = self.Conv1(x)
        x = F.relu(x)
        x = self.pooling(x)
        x = self.Conv2(x)
        x = F.relu(x)
        x = self.pooling(x)
        x = x.view(-1, 12*4*4)
        output = self.fullyConnected1(x)
        return output

# sample of each class
def sample_of_classes(dataset):
    classes = ['0', '1', '2', '3', '4', '5', '6', '7', '8', '9']
    plt.figure(figsize=(10, 5))
    for k in range(len(classes)):
        plt.subplot(2, 5, k + 1)
        sample_image = dataset.data[dataset.targets == k][0]
        plt.imshow(sample_image.squeeze().numpy(), cmap='gray')
        plt.title(f'Class {classes[k]}')
        plt.axis('off')
    plt.tight_layout()
    plt.show()

# train the NN
```

```

def train(model, device, epoch, trainData_loader, optimizer):
    model.train() # train model
    train_loss = 0
    for batch_idx, (data, target) in enumerate(trainData_loader): # data -
-> mini-batch
        data, target = data.to(device), target.to(device)
        optimizer.zero_grad()
        output = model(data) # predictions based on input data
        loss = criterion(output, target) # loss -> predictions vs real
        loss.backward() # gradients
        #print(f"Gradient, epoch {epoch}, mini-batch
{batch_idx}:\n{model.Conv1.weight.grad}")
        optimizer.step() # refresh weights, from gradients
        train_loss += loss.item()
        if batch_idx % 100 == 0:
            print('Training epoch: {} [{} / {}] {:.0f}%]\tloss:
{:.6f}'.format(
                epoch, batch_idx * len(data),
len(trainData_loader.dataset),
                100. * batch_idx / len(trainData_loader), loss.item()))
            return train_loss / len(trainData_loader)

# check how it works after training
def validate(model, device, valData_loader):
    # for 100%-train & 0%-validate
    if valData_loader is None:
        print("No validation data")
        return 0.0

    model.eval() # no train
    val_loss = 0
    with torch.no_grad(): # no gradients cal
        for data, target in valData_loader:
            data, target = data.to(device), target.to(device)
            output = model(data) # predictions based on input data
            val_loss += criterion(output, target).item() # cal loss

    # val_loss / num of batches
    val_loss /= len(valData_loader) if len(valData_loader) > 0 else 1
    print('\nAV loss: {:.4f}\n'.format(val_loss))
    return val_loss

# accuracy
def accuracy_ConfMatrix(model, device, testData_loader):
    model.eval()
    total_number = 0 # total num of examples
    correct_predictions = 0 # correct predictions
    Real_targets = []
    Predictions = []

    with torch.no_grad():
        for data, target in testData_loader: # data , "real" labels -->
save
            data, target = data.to(device), target.to(device)
            outputs = model(data) # data through model
            _, predicted = torch.max(outputs.data, 1) # highest /
predicted
            total_number += target.size(0) # ++

```

```

        correct_predictions += (predicted == target).sum().item() #
num of correct predictions --> add to correct_predictions
        Real_targets.extend(target.cpu().numpy()) # real "labels"
        Predictions.extend(predicted.cpu().numpy()) # data

    accuracy = correct_predictions / total_number
    print('Accuracy of the Neural Network: {:.2f}%'.format(accuracy *
100))

    ConfMatrix = confusion_matrix(Real_targets, Predictions)
    return accuracy, ConfMatrix

if __name__ == "__main__":

    # load & transform dataset
    transform = transforms.Compose([
        transforms.ToTensor(),
        transforms.Normalize((0.5, ), (0.5, ))
    ])

    # load dataset
    dataset = datasets.MNIST(root='../TheData', train=True, download=True,
transform=transform)

    sample_of_classes(dataset)

    # train : 80% , validation : 100 - 80 = 20%

    #training = int(0.05 * len(dataset))
    #training = int(0.1 * len(dataset))
    #training = int(0.5 * len(dataset))
    #training = int(1 * len(dataset))
    training = int(0.8 * len(dataset))
    validation = len(dataset) - training
    training_subset, validation_subset = random_split(dataset, [training,
validation])

    print(f"Training: {len(training_subset)}")
    print(f"Validation: {len(validation_subset)}")

    # Dataloaders
    trainData_loader = DataLoader(training_subset, batch_size=batch_size,
shuffle=True)
    valData_loader = DataLoader(validation_subset, batch_size=batch_size,
shuffle=False)
    best_val_loss = float('inf')

    model = NeuralNetwork()
    criterion = nn.CrossEntropyLoss()
    optimizer = optim.SGD(model.parameters(), lr=learning_rate) #
Stochastic Gradient Descent
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    # save train , val losses
    training_losses = []
    validate_losses = []

```

```

# initialize losses to compare
best_training = float('inf')
best_validate = float('inf')

# train model for each epoch
for epoch in range(1, num_epochs + 1): # 1 to 10
    train_loss = train(model, device, epoch, trainData_loader,
optimizer) # av. loss during training in every epoch
    val_loss = validate(model, device, valData_loader) # av. loss
during validation in every epoch

    # compare current epoch with previous and save the epoch with <
loss
    if train_loss < best_training:
        print('Training loss decreased from {:.4f} to
{:.4f}'.format(best_training, train_loss))
        best_training = train_loss
        torch.save(model.state_dict(), 'best_training.pth')

    if val_loss < best_validate:
        print('Validation loss decreased from {:.4f} to
{:.4f}'.format(best_validate, val_loss))
        best_validate = val_loss
        torch.save(model.state_dict(), 'best_validate.pth')

    # adding losses in list
    training_losses.append(train_loss)
    validate_losses.append(val_loss)

test_dataset = datasets.MNIST(root='../Thedata', train=False,
download=True, transform=transform)
testData_loader = DataLoader(test_dataset, batch_size=batch_size,
shuffle=False) # test data in batches

plt.figure(figsize=(10, 7))
plt.plot(range(1, num_epochs + 1), training_losses, label='training
loss', color='purple')
plt.plot(range(1, num_epochs + 1), validate_losses, label='validate
loss', color='orange')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.title('training & validation losses')
plt.legend()
plt.show()

accuracy, ConfMatrix = accuracy_ConfMatrix(model, device,
testData_loader)
print(f"Accuracy: {accuracy}")
print(f"Confusion Matrix:\n{ConfMatrix}")

```

Μέρος Β : extract_features.py

```
import numpy as np
from skimage.feature import hog
import torch
from torchvision import datasets, transforms
import torch.nn as nn
import torch.optim as optim
import matplotlib.pyplot as plt
from torch.utils.data import TensorDataset, DataLoader, random_split
from sklearn.metrics import confusion_matrix

def extract_features(data):
    features = [] # save features
    for image in data:
        image = image.numpy() # image --> numpy array
        # 4 x 4 pixel , # 2 x 2 block (4 cells)
        hog_feature, hog_image = hog(image, pixels_per_cell=(4, 4),
        cells_per_block=(2, 2), visualize=True, block_norm='L2-Hys')
        features.append(hog_feature) # add hog features
    return np.array(features)

class NeuralNetwork(nn.Module):
    def __init__(self):
        super(NeuralNetwork, self).__init__()
        self.fc1 = nn.Linear(1296, 128) # input 1296 HOG features -->
        output 128
        self.fc2 = nn.Linear(128, 10) # 10 classes output

    def forward(self, x):
        x = torch.relu(self.fc1(x))
        x = self.fc2(x)
        return x

def train_model(model, train_loader, val_loader, criterion, optimizer,
num_epochs=10):
    best_val_loss = float('inf')
    training_losses = []
    validate_losses = []

    for epoch in range(num_epochs):
        model.train() # train model
        train_loss = 0
        for features, targets in train_loader:
            optimizer.zero_grad()
            outputs = model(features) # predictions based on input data
            loss = criterion(outputs, targets) # loss -> predictions vs
            real
            loss.backward() # gradients
            optimizer.step() # refresh weights, from gradients
            train_loss += loss.item()

        train_loss /= len(train_loader)
        training_losses.append(train_loss)
        print(f'Epoch [{epoch+1}/{num_epochs}], Train Loss:
        {train_loss:.4f}')
```



```

        val_loss = validate(model, val_loader, criterion)
        validate_losses.append(val_loss)
        print(f'Epoch [{epoch+1}/{num_epochs}], Validation Loss:
{val_loss:.4f}')

        if val_loss < best_val_loss:
            best_val_loss = val_loss
            torch.save(model.state_dict(), 'hog_model.pth')

    print(f'Best Loss: {best_val_loss:.4f}')

    plt.figure(figsize=(10, 7))
    plt.plot(range(1, num_epochs + 1), training_losses, label='Training
Loss', color='pink')
    plt.plot(range(1, num_epochs + 1), validate_losses, label='Validation
Loss', color='grey')
    plt.xlabel('Epochs')
    plt.ylabel('Loss')
    plt.title('Training & Validation Losses')
    plt.legend()
    plt.show()

def validate(model, val_loader, criterion):
    # 100% train & 0% validate
    if val_loader is None:
        print("No validation data")
        return 0.0

    model.eval() # no train
    val_loss = 0
    num_batches = len(val_loader)
    with torch.no_grad(): # no gradients cal
        for features, targets in val_loader:
            outputs = model(features) # predictions based on input data
            val_loss += criterion(outputs, targets).item() # cal loss
    # val_loss / num of batches
    val_loss /= num_batches if num_batches > 0 else 1
    print(f'Validation Loss: {val_loss:.4f}')
    return val_loss

def accuracy_ConfMatrix(model, device, test_loader):
    model.eval()
    total_number = 0 # total num of examples
    correct_predictions = 0 # correct predictions
    all_targets = []
    all_predictions = []

    with torch.no_grad():
        for data, target in test_loader: # data , "real" labels --> save
            data, target = data.to(device), target.to(device)
            outputs = model(data) # data through model
            _, predicted = torch.max(outputs.data, 1) # highest /
predicted
            total_number += target.size(0) # ++
            correct_predictions += (predicted == target).sum().item() #
num of correct predictions --> add to correct_predictions
            all_targets.extend(target.cpu().numpy()) # real "labels"

```

```

        all_predictions.extend(predicted.cpu().numpy()) # data

    accuracy = correct_predictions / total_number
    print('Accuracy of the Neural Network: {:.2f}%'.format(accuracy *
100))

    ConfMatrix = confusion_matrix(all_targets, all_predictions)
    return accuracy, ConfMatrix


if __name__ == "__main__":

    transform = transforms.Compose([transforms.ToTensor()]) # --> tensor
    training_dataset = datasets.MNIST(root='../TheData', train=True,
download=True, transform=transform) # load
    testing_dataset = datasets.MNIST(root='../TheData', train=False,
download=True, transform=transform) # load

    train_data_image = training_dataset.data # image from the train
dataset
    test_data_image = testing_dataset.data # image from the test dataset

    # extract features from the images (training & test)
    train_features = extract_features(train_data_image)
    np.save('train_features.npy', train_features)
    print(f"Train Features Shape: {train_features.shape}")
    train_features = np.load('train_features.npy')

    test_features = extract_features(test_data_image)
    np.save('test_features.npy', test_features)
    print(f"Test Features Shape: {test_features.shape}")
    test_features = np.load('test_features.npy')

    # --> tensors
    train_features_tensor = torch.tensor(train_features,
dtype=torch.float32)
    test_features_tensor = torch.tensor(test_features, dtype=torch.float32)

    # TensorDataset : features (inputs) , targets (outputs)
    train_dataset = TensorDataset(train_features_tensor,
training_dataset.targets)
    test_dataset = TensorDataset(test_features_tensor,
testing_dataset.targets)

    # 80% : training and 20% : validate
    dataset_size = len(train_dataset)
    #train_size = int(0.05 * dataset_size)
    #train_size = int(0.1 * dataset_size)
    #train_size = int(0.5 * dataset_size)
    #train_size = int(1 * dataset_size)
    train_size = int(0.8 * dataset_size)
    val_size = dataset_size - train_size
    train_dataset, val_dataset = random_split(train_dataset, [train_size,
val_size])

    train_loader = DataLoader(train_dataset, batch_size=64, shuffle=True)

```

```

val_loader = DataLoader(val_dataset, batch_size=64, shuffle=False)
test_loader = DataLoader(test_dataset, batch_size=64, shuffle=False)

model = NeuralNetwork()
criterion = nn.CrossEntropyLoss()
optimizer = optim.SGD(model.parameters(), lr=0.01, momentum=0.9) #
Stochastic Gradient Descent algorithm
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

train_model(model, train_loader, val_loader, criterion, optimizer,
num_epochs=10)

accuracy, ConfMatrix = accuracy_ConfMatrix(model, device,
test_loader)
print(f'Confusion Matrix:\n{ConfMatrix}')

```

Μέρος Α αποθήκευση καλύτερου δικτύου : BestModel.py

```

import torch
import torch.nn as nn
import torch.nn.functional as F

# use the same NN
class NeuralNetwork(nn.Module):
    def __init__(self):
        super(NeuralNetwork, self).__init__()
        self.Conv1 = nn.Conv2d(in_channels=1, out_channels=6,
kernel_size=5, padding=0)
        self.pooling = nn.MaxPool2d(kernel_size=2, stride=2)
        self.Conv2 = nn.Conv2d(in_channels=6, out_channels=12,
kernel_size=5, padding=0)
        self.fullyConnected1 = nn.Linear(in_features=12*4*4,
out_features=10)

    def forward(self, x):
        x = self.Conv1(x)
        x = F.relu(x)
        x = self.pooling(x)
        x = self.Conv2(x)
        x = F.relu(x)
        x = self.pooling(x)
        x = x.view(-1, 12*4*4)
        output = self.fullyConnected1(x)
        return output

# new model
Best_model = NeuralNetwork()

# adjust the weights from the .pth files (the best performance / less loss)
state_dict_training = torch.load('best_training.pth')
state_dict_validate = torch.load('best_validate.pth')

Best_model.load_state_dict(state_dict_training)

print("Conv1.weight:", state_dict_training['Conv1.weight'])

```

```
print("Conv1.bias:", state_dict_training['Conv1.bias'])
print("Conv2.weight:", state_dict_training['Conv2.weight'])
print("Conv2.bias:", state_dict_training['Conv2.bias'])
print("fullyConnected1.weight:",
state_dict_training['fullyConnected1.weight'])
print("fullyConnected1.bias:",
state_dict_training['fullyConnected1.bias'])
```